
Reproducible Science: Machine-Checkable Claims from Preregistration to Publication

Elliot Tower
elliott@elliotttower.ai

Abstract

Every claim in a paper comes from somewhere. AI tooling allows us to quickly generate hypotheses, run analyses, and draft manuscripts, but we identify verification and provenance as open issues: (1) verifying that the finished paper agrees with the artifacts behind it typically requires manual or agentic verification, which is expensive and difficult to audit; (2) verification usually does not include a denominator for the number of sources or artifacts checked; (3) verification snapshots quickly become stale, and are not guaranteed to find every issue or reproduce the same issues on subsequent runs; (4) more experiments means more researcher degrees of freedom and post-hoc analysis, even if unintentional, or done by an agent; and (5) verification is typically done post-hoc, and does not cover all steps of the research lifecycle.

We present Reproducible Science¹, a declarative, continuous, machine-checkable provenance layer for research claims, throughout the whole research lifecycle. `prereg` freezes a plan before the run, `results` seals its inputs, records its outputs and binds numbers in the manuscript to the run that produced them, `citations` pins exact quotations inside cited sources, and `repro` checks every declaration against the artifact it names. Our Claude Code plugin records provenance at the time of experimentation and reports drift on every edit through hooks. Applying this contract to this paper surfaced two errors, one of which had already gone stale twice. As agents take over more of the research lifecycle, the artifacts they leave behind are worth only as much as we can verify the claims inside them.

1 Introduction

Surveys of research agents name verification as the step that has not kept pace [Ding et al., 2026]. Re-running the analysis is the usual answer, and it is usually unavailable: across 863,878 attempted executions of valid notebooks, 24.11% ran without errors and 4.03% produced the results the notebook recorded [Pimentel et al., 2019]. The deeper problem is that re-running checks the wrong object. A notebook that re-executes perfectly can still sit beside a manuscript printing a value no output contains, because execution establishes that the code runs and says nothing about what the paper claims the code produced. Reproducibility and verification are different properties. A project can preserve its code, its data and its logs and still print a value no output contains, quote a passage no extractor read, or call an analysis confirmatory after its plan changed.

We call what accumulates **verification debt**. Like debt in software, it is cheapest to detect when it is introduced and expensive to reconstruct afterwards. At the moment a value changes, the run that produced the old one is still on disk, the source still resolves, and the edit carries an author and a commit. Later the run has been overwritten, the source was fetched once without its address recorded, and which document a citation referred to is no longer recoverable from the citation.

¹Code and artifacts: <https://github.com/elliotttower/reproducible-science-paper>

Table 1: Three boundaries of a research project, the failure each admits when nothing checks it, and the comparison this system performs. All three are decided by a digest and a comparison rule over declared addresses, which is why one engine serves them.

boundary	what goes wrong when it is unchecked	what the checker compares
plan to run	a hypothesis or an analysis is rewritten after the outcome is visible	a run record against a plan pinned by digest
run to manuscript	a printed value drifts from the output it was read from	a declared address in a sealed result file
source to manuscript	a quotation or an identifier does not occur in the artifact cited for it	a passage against a content-addressed source

Most of what a reader would want checked is not mechanically checkable: whether a design answers the question, whether a conclusion follows. A narrow part is. A number printed in a manuscript either matches the results file it came from or does not, and a quoted sentence either occurs in the cited source or does not. That part can be settled without a human and without a model, at a cost small enough to run on every commit rather than once before submission.

Nothing failed because nothing ran. In this implementation a missing extractor, the program that pulls text out of a PDF, returned the empty string, the verifier recorded an unreadable document, and the run summary read *nothing failed*. A check that read no text and a check that read the text and found it correct produced the same line. A separate incident is worse: a script collected the digests cited by every registration under one directory and verified each against a file; the directory was renamed, the glob matched nothing, the script exited zero, and continuous integration, a pre-push hook and a make target were all green in that state. A report that returns success is not evidence that anything was examined, and an outcome vocabulary with no term for non-execution cannot say so.

Position. Three claims follow, each against a live alternative. *Prospective decisions require prospective records*: a frozen plan cannot establish that an author was honest, but it makes subsequent rewriting detectable and separates confirmatory work from exploratory work. *Evidence requires declared addresses*: a reported 0.42 should name the pointer or selector it came from, because searching afterwards for a numerically similar value is not provenance and does not scale. The alternative is recovery, and a contemporaneous system takes it: Paperclip binds the same unit, a claim to a location in a source, and verifies it with a language-model subagent against line ranges into its own re-parse [Generative Expert Labs, 2026]. A verdict from a model is not recomputable from the artifact, and an address into a re-parse moves when the parser improves. We make the opposite choice on both. *Verification requires an explicit non-execution state*: a checker that cannot distinguish matched, disagreed, absent and could-not-run has no way to report that a green build examined nothing. Our own infrastructure violated the non-execution requirement twice.

Contributions. (1) **Lifecycle verification**: a model spanning the plan-to-run, run-to-manuscript and source-to-manuscript boundaries of Table 1, designed to run while a project changes. (2) **Evidence assertions**: typed assertions binding manuscript values and quotations to declared locations in content-addressed artifacts, with four implemented variants over a shared engine and report. (3) **Staged outcomes**: a deterministic model separating execution, extraction and comparison, with project policy applied to the resulting facts rather than embedded in the verifier. We instantiate all three in a deployed toolkit and evaluate them through conformance tests, fault injection, a content-addressed quotation audit, defects met in ordinary use, and an audit of a project developed using this toolkit.

2 Model and Contract

Section 1’s two incidents fail differently — one check never ran, the other ran and read no text — and both reported what a passing check reports. The three boundaries of Table 1 share one architecture: declare an address, pin the artifact it points into, compare deterministically, and report each stage separately. Two of the three are decided by an *evidence assertion*, which binds a manuscript claim to an artifact, a locator within it, and the value or passage expected there; the plan-to-run boundary is decided by reading a run record beside a pinned plan, so no evidence assertion is defined for it. Four

variants are implemented across the other two boundaries. They differ in how evidence is located and compared, and share artifact pinning, staged outcomes and one report.

Table 2: The four implemented variants. A table cell is addressed by a selector over key columns rather than by row index, because an index silently addresses a different cell when a table is reordered. A selector matching more than one row is reported rather than resolved to the first. A correspondence names no expected value in the manifest and neither side is the reference, so a disagreement prints both readings and no field can be edited to silence it.

kind	assertion	located by	compared how
metric	this artifact holds this value here	RFC 6901 JSON Pointer	decimal, at the precision the manuscript printed
table	this table holds this value in this cell	column plus a row selector	decimal, at the precision the manuscript printed
quote	this passage occurs in this artifact	page, optionally	verbatim after normalization that preserves which words appear
correspondence	two artifacts hold the same value	a locator on each side, prose anchors or a pointer	decimal, at the precision the manuscript printed

Three stages. A check must execute, extract, and compare, and the three fail independently. Table 3 gives the outcomes and the flattened view derived from them. The rows differ in what a reader should do: revise the claim, fix the pointer or run the analysis that produces the value, install a package, file a bug. A report that cannot distinguish them directs none of those actions.

Table 3: The three stages and the flattened view derived from them, the single label a report prints. A results file with no `bootstrap_n` key declines to assert that the value differs from the manuscript's; it asserts nothing about it. Backend exceptions are not interchangeable either: an unavailable backend yields `unchecked`, and any other exception yields `error` and never `unchecked`.

situation	execution	extraction	comparison	flattened
value present	completed	extracted	match	verified
value differs in a readable artifact	completed	extracted	mismatch	mismatch
pointer does not resolve	completed	absent	—	not_found
extractor not installed	unavailable	not attempted	—	unchecked
backend defect	failed	not attempted	—	error

Pins. Artifacts are content-addressed, and pins are resolved before any evidence is read, so a source that moved is known before decisions are computed against it. Decisions against a broken or unpinned artifact still run and are reported, marked non-authoritative, because a diagnostic is more useful than a blank.

Facts, then policy. Verification returns facts. Whether they constitute a pass depends on what the project is for: an unchecked citation is acceptable in a draft and disqualifying in a submission, and both readings are correct about the same run. The engine therefore reports the three stages and computes no verdict, and a profile maps outcomes to severities. Re-grading under a stricter profile re-reads the stored report and executes nothing, so a submission is judged on the facts the draft was checked with rather than on a second run that may not reproduce them.

3 Implementation and Workflow

The checker runs on ordinary files. It does not require an experiment to be conducted in a particular language, notebook system, analysis framework or agent runtime, and it does not require a manuscript to be authored inside its toolchain. The fifteen papers in the quotation audit were written without this contract in mind, and none was modified to be auditable. A value is read from JSON,

Table 4: Policy profiles over identical facts. Every profile requires that at least one assertion was evaluated, because a project with no evidence anywhere otherwise satisfies every condition trivially. `not_offered` marks a claim that offers no evidence.

profile	mismatch	not_found	unchecked	not_offered	unpinned
exploratory	error	warning	warning	ignore	ignore
publication	error	error	warning	warning	warning
publication, confirmatory claims	error	error	error	error	—
strict	error	error	error	error	error

delimited text, a workbook, Parquet, Stata, SPSS, R data, HDF5 or NetCDF, and a quotation from a PDF, plain text, a workbook, a Word document or JATS XML.

A project declares as it goes. A plan is frozen before the run it governs, against a commit and a content hash. Inputs are sealed before an analysis and outputs recorded after it. A sentence that prints a number declares the address the number came from, at the time the sentence is written. Verification then reads the declarations and reports on each.

Table 5: The four commands and the boundary each decides. The first three write declarations and the fourth reads them, applying a policy to the result.

command	what it establishes
<code>prereg</code>	that the plan which ran is the plan that was frozen, against a commit and a content hash
<code>results</code>	that a reported number came from a recorded run over sealed inputs
<code>citations</code>	that a quoted passage occurs in the source cited for it
<code>repro</code>	that every declared assertion holds against its pinned artifact, and which do not

A worked example. Appendix A reports 113 unresolved quotations, and this paper’s manifest binds that sentence with two assertions: a quote assertion checking that the manuscript still prints it, and the `metric` assertion below, checking that the pinned corpus artifact holds the number at the declared pointer.

```
- kind: metric
  artifact: corpus          # pinned: sha256 4bdb0228...
  pointer: /per_reader/poppler-layout/outcomes/not found
  reported: '113'
```

`repro verify` reports `verified` for both. The Claude Code plugin prompts for such declarations while the artifacts are produced: after each edit it names any number or quotation in a manuscript that nothing yet backs, and before an analysis runs it says when the governing plan is unfrozen, never blocking the work it observes. A companion mod shows the state of a project’s records in the session, one row per command, with a warning when a frozen plan was edited or a run was recorded with nothing sealed (Appendix B, Figure 1).

Preregistration is optional and per claim. A claim is marked confirmatory, exploratory, or not applicable, and only the first requires a plan behind it. This matters for the honesty of the report: a project that registers nothing is not failed for it, and a project that calls a claim confirmatory is held to the plan it named.

4 Evaluation

The conformance suite and the fault injections are in Appendix A: 25 fixtures, each recording its expected outcome, reason and artifact validity as a triple so a case cannot pass by reaching the right verdict for the wrong cause, and four injected faults each distinguishing a class the flattened view would otherwise merge.

4.1 Self-audit

This paper is an instance of what it describes. Every number it reports about its extractor comparison and its independent-repository audit carries two assertions: that the manuscript states the sentence, and that a pinned artifact holds the value at a declared pointer. Seventeen claims are bound this way, 34 assertions over seven artifacts, and they pass under the strictest policy profile. The manifest and the artifacts it pins are in the repository, so the check is one a reader runs rather than a result they take on trust.

Building the manifest surfaced two defects in this paper’s own numbers. The four union counts in Section 4.2 (both poppler modes, any extractor, no extractor, and what the low-level parser alone adds) had been computed correctly and written to no file, leaving nothing that could re-check them; derived into an artifact, all four matched what the manuscript printed. The conformance-suite count had the opposite problem, recorded but stale: a stored figure gave nineteen fixtures where the suite holds 25, and an earlier documentation audit had already found the specification stating eighteen against a probe that counted nineteen, so the count had gone stale twice before an inventory re-counted it category by category.

Two limits. This is an integration demonstration rather than independent validation: the manuscript, the manifest and the engine share an author. And the counts in this subsection are not themselves asserted, because a claim about the number of assertions changes that number.

4.2 Quotation audit

Six extractors read the same 2,364 passages out of the same pinned bytes and returned six different answers, from 2,314 down to 1,716. A verification snapshot therefore records the reader as much as the corpus, and a rerun under a different toolchain is not guaranteed to find the same issues. Under poppler’s layout mode 113 quotations fail to resolve and every failure traces to extraction (Appendix A), so a checker with only pass and fail would have accused the manuscripts 113 times with no misquotation present.

Table 6: Passages resolved per extractor over one corpus, and the cost of each. The two poppler modes fail in opposite directions and together reach 2,360. Ranking by total resolved is a poor guide to which extractors to consult: the low-level parser resolves 598 fewer passages than the best single extractor and is the only one that extends the union, reaching 2,362. Two quotations resolve under no extractor.

extractor	resolved of 2,364	seconds
poppler, reading order	2,314	6.9
poppler, layout	2,250	6.9
pure-Python reader	2,229	43.9
rendering-engine binding	2,092	4.8
structural extractor	1,973	6.2
low-level parser	1,716	2.7
both poppler modes	2,360	13.8
all six	2,362	71.4

4.3 Recall against a benchmark

The corpora above carry no ground truth: nobody has labelled which of a paper’s numbers its artifact really holds, so a located value is evidence of an occurrence. CORE-Bench supplies the missing half. Each capsule, CORE-Bench’s package of a paper’s code and data, names the values its analysis produces, so recall is measured against a stated answer set rather than estimated [Siegel et al., 2024].

Over 45 capsules the benchmark states 240 values, of which 146 are located: a recall of 61%. Table 7 gives it by the strength of the value, where strength is the number of constraining digits. Recall falls as values carry more digits and then rises again, because the tiers differ in what they are made of rather than in difficulty: a two-digit integer is found in almost any artifact and a long decimal is found only where it was written.

Table 7: Recall over 45 CORE-Bench capsules, against the values each capsule states it produces. Eight capsules hold a record no reader here opened, so their misses are `unchecked` rather than `absent`: a two-state report would count all of them against the capsule. This is the same separation Section 4.2’s quotation audit makes, on a third party’s corpus and against a known answer set.

strength	stated	located	recall
trivial	59	54	92%
weak	76	40	53%
moderate	44	15	34%
strong	61	37	61%
all	240	146	61%

5 Findings from Deployment

5.1 Defect classes surfaced in ordinary use

Running the checker over our own corpus during ordinary research surfaced six defect classes. Table 8 lists them. Each is reported as a class that occurs and is mechanically detectable; no frequency is claimed, and none of these counts enters any rate in Section 4. Two of the six are defects in a check rather than in a manuscript, which is the case the three-stage model exists for.

Table 8: Six defect classes surfaced by running the checker over one author’s corpus. Status records whether a change has been made that prevents the class, not whether the instance was repaired.

class	what is observably wrong	status
stored value disagrees with the printed value	nine of ten table rows disagree with the file the row is read from	open
false mismatch under float equality	three of thirty-nine agreements hold only under decimal comparison	closed
recorded hash compared against nothing	digests recorded and never compared to the bytes on disk	closed
check that runs on no files	a glob matches nothing, the check exits zero, and the build is green	open
claims separated from the evidence they name	178 assertions marked verified, and no source in any commit	closed locally
reported figure that stopped reproducing	a reported count no longer reproduces from the bibliography it described	open

Two that integrity checking cannot reach. In the first row every artifact resolves at its recorded digest, so no integrity check fires: the files hold the values the run wrote, and the manuscript’s table holds different values. A value printed at 0.001 is stored as exactly zero, and one printed at 0.0003 as 9.73e-7. The disagreement is reachable only by a check that compares a file to the sentence citing it. In the fifth, a repository split carried 178 assertions and left the sources they named behind; the stored `verified` marks survived the split, so the repository reported a fully verified claim set that a reader could recheck in no part. Quotation verification had passed over every one of those sources, because a quotation resolves against the text as it stands.

5.2 What a reader can re-check from a clone

We ran the four commands over a clone of the public repository for a project developed using this toolkit, at a pinned commit.² The project keeps registered plans, a results ledger and pinned quotations, and declares no evidence manifest. The clone carries 1,267 tracked files, three registered plans, a ledger of 153 events binding 131 manuscript claims to 13 runs, and seven claim records holding 18 quotations. Table 9 gives what each command established from the clone alone and what it could not.

²<https://github.com/elliotttower/knockout-epistasis-dynamics>, commit cb82384.

Table 9: Four commands over a clone of the project’s public repository. Two establish their result from the clone alone, because the plans and the ledger are committed with it. The quotation check reads no source and reports that, where a two-state report would have to print a pass or a failure.

command	what it found	what it could not establish
<code>prereg</code>	three plans, each unchanged against the digest recorded when it was frozen	whether a plan was written before the outcomes it governs were seen
<code>results</code>	the chain is intact and anchored at 153 events; 8 of 12 recorded files match their digest, and 4 carry more than one recorded digest	what share of the manuscript’s numbers the 131 claims cover: the manuscript is not in the repository
<code>citations</code>	18 quotations declared against 7 sources, each carrying a SHA-256; none checked, because the sources are not in the repository	whether any quotation occurs in its source
<code>repro</code>	no evidence manifest; exit 2, nothing addressed	any assertion, none having been declared

The plans and the ledger verify from a clone because both are committed with the repository. The quotations do not: a publisher’s PDF is not the author’s to redistribute, so the repository carries each source’s digest and not the source, and all 18 quotations come back unchecked.

5.3 An edit to a registered plan

A preregistration carries a recorded digest, and an edit to it is detected by recomputing that digest rather than by asking the author what changed. In ordinary use `prereg check` caught a subagent that had edited a frozen plan partway through an analysis, and the edit was reverted. The hook that watches for it fires on every edit and never refuses one, because a check that can halt the work it observes gets switched off. Amending a registration is legitimate; amending it silently is what the digest makes impossible.

5.4 Use across a corpus

The commands have been run while the work was being done rather than over a corpus assembled to be audited. Twenty-two repositories carry 466 claim files declaring 5,590 quotations. Twenty-three preregistered plans across five repositories carry a freeze, and every one still verifies against the digest recorded when it was frozen. A sweep of 25 repositories found 21 where at least one command applied and four declaring nothing for any of them, which the report states rather than counting as a pass.

Cost is what makes the check a commit-time one. Verifying 12 to 34 assertions over three to seven artifacts takes 0.83 to 0.91 seconds; resolving 61 quotations against 12 content-addressed sources takes 3.69 seconds.

6 Related Work

Claim-to-location binding. Paperclip binds individual claims to individual locations in sources: an agent adds a claim naming a document identifier and a line range, and a commit verifies each claim against the source text, dropping unsupported claims, with a log for audit [Generative Expert Labs, 2026]. The unit is the same as ours and three properties differ. Verification is performed by a language-model subagent, so the verdict is not reproducible from the artifact. Line ranges address the system’s own parse of a document, which is refetched and re-read, so an address points into a mutable representation and improving the parse moves every stored range. No digest is recorded, which under Section 2 leaves every such artifact unpinning and every decision against it non-authoritative. A commit also lands when verification is skipped, which is the condition Section 1 describes.

Repository auditing. `adduce` audits a research repository against 78 rules and returns a score, asking whether “for every number in the paper, can I point to the artifact that produced it, and will that artifact still produce it elsewhere?” [Chudasama, 2026], which is this paper’s question reached independently. The agreement runs deeper: a rule there returns one of five statuses, and two of them,

unknown and not-applicable, are excluded from the score rather than counted as passes. The unit differs, and so does where an outcome comes from. adduce grades a repository and a rule author selects among five flat statuses, where one assertion here binds one printed value to one declared address, no score is computed, and the outcome follows from which of the three stages stopped. The two compose, and the implementation registers its verification as a rule adduce runs.

Artifact-level verification. Code Ocean verifies a capsule before publication and states that the check establishes mechanical reproducibility, a user pressing run and obtaining results, and separately that it does not check whether a capsule’s results match the article it accompanies [Code Ocean, 2026]. Whole Tale packages code, data, methods and environment with the narrative as a re-instantiable tale [Brinckman et al., 2019]. In both, the unit of citation and the unit of verification are the same object, so the outcome is one property of the whole. Here the unit of verification is finer than the unit of citation, and a report states which of a manuscript’s values an artifact still supports rather than whether the artifact runs.

Entailment, markup and write-time binding. FEVER, SciFact and CliniFact predict a label given retrieved text and score it against human annotation [Thorne et al., 2018, Wadden et al., 2020, Zhang et al., 2025], and ClaimReview binds claim text to the work it appeared in while recording no digest and enumerating no outcomes [Schema.org, 2026]. Our outcomes name what a byte or decimal comparison found. Literate programming generates the document from the execution producing its values [Leisch, 2002], which removes the drift, requires the manuscript to be authored inside the toolchain, and absorbs a change where this reports one.

Abstention as an outcome. Fellegi and Sunter classify a candidate pair as match, non-match, or possible match held for review, and prove that rule minimizes the review region at fixed error bounds [Fellegi and Sunter, 1969]. The three-way structure, with abstention as a first-class outcome rather than a failure, is the shape Table 3 takes. Their third state is reached probabilistically from estimated agreement patterns; ours is reached because an extractor was absent or a pointer did not resolve.

Verification in other fields. Metrology defines verification as evidence that specified requirements are met, and keeps verification apart from validation, which asks whether those requirements suit an intended use [Joint Committee for Guides in Metrology, 2012]; this system performs verification. The philosophy of measurement separates an instrument’s reading from the result inferred from the reading [Tal, 2017], and an extractor is an instrument in this sense: in Section 4.2, six extractors read the same bytes and returned six results. In formal verification, many early automated theorem provers performed proofs without recording them, so a result could not be checked afterwards [MacKenzie, 2001]; here every verdict is stored with the pinned artifacts it was computed against.

7 Limitations

The extractor is trusted. Content addressing establishes that an artifact’s bytes did not change and nothing about whether the extractor read them correctly. A layout parser that splices a value from the wrong column, or an optical pass that reads 0.9489 as 0.9439, leaves the digest intact and returns a confident answer. The blockchain literature calls this the oracle problem [Albizri and Appelbaum, 2021]. Recording the extractor’s identity and a digest of the text it produced makes a changed reading visible, which addresses drift and not correctness.

Registration establishes order, not honesty. A run record states when a plan was registered and when the run began, and the check establishes that those two are consistent. It does not establish that the registration is contemporaneous with what it claims. An outside timestamp [Adams et al., 2001] bounds that: `prereg` and `results` timestamp a plan’s digest and the ledger’s head through OpenTimestamps [Todd, 2016], which fixes the latest date at which each could have been written. A timestamp cannot show that a plan was not written earlier and withheld.

Declaration is authored, and coverage is gameable. Only declared numbers are checked, so a project can pass by declaring only the numbers it expects to match. We state coverage alongside outcomes so that a reader can see what fraction of the manuscript’s numbers was checked.

8 Conclusion

Re-running an analysis establishes that the code runs. Verification debt accumulates in the difference between that and what a manuscript says it produced. Values, quotations and registered plans differ in how each is extracted and little else, so one engine reports all three. The three stages exist because disagreement, absence and non-execution have different remedies, and because collapsing them produced a run that examined nothing.

References

- Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). Request for Comments 3161, Internet Engineering Task Force, 2001. URL <https://www.rfc-editor.org/info/rfc3161>.
- Abdullah Albizri and Deniz Appelbaum. Trust but Verify: The Oracle Paradox of Blockchain Smart Contracts. *Journal of Information Systems*, 35(2):1–16, 2021. doi: 10.2308/isis-19-024. URL <https://doi.org/10.2308/isis-19-024>.
- Adam Brinckman, Kyle Chard, Niall Gaffney, Mihael Hategan, Matthew B. Jones, Kacper Kowalik, Sivakumar Kulasekaran, Bertram Ludäscher, Bryce D. Mecum, Jarek Nabrzyski, Victoria Stodden, Ian J. Taylor, Matthew J. Turk, and Kandace Turner. Computing environments for reproducibility: Capturing the “Whole Tale”. *Future Generation Computer Systems*, 94:854–867, 2019. doi: 10.1016/j.future.2017.12.029. URL <https://doi.org/10.1016/j.future.2017.12.029>.
- Harshil Chudasama. adduce: a local research-artifact auditor, 2026. URL <https://github.com/QHarshil/adduce>. Software, version 0.1.2, released 2026-08-04. Author and title taken from the project’s own CITATION.cff.
- Code Ocean. Code Ocean’s verification process for computational reproducibility & quality, 2026. URL <https://docs.codeocean.com/osl-guide/publishing-on-code-ocean/the-verification-process/code-oceans-verification-process-for-computational-reproducibility-and-quality>. Open Science Library documentation. Read 2026-08-27.
- Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, and Ling Zhang. Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap, 2026. URL <https://arxiv.org/abs/2608.05179>.
- Ivan P. Fellegi and Alan B. Sunter. A Theory for Record Linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969. doi: 10.1080/01621459.1969.10501049. URL <https://doi.org/10.1080/01621459.1969.10501049>.
- Generative Expert Labs. Paperclip: Agent-native indexing of scientific literature and data, 2026. URL <https://paperclip.gxl.ai/>. Software. CLI client at <https://github.com/GXL-ai/paperclip> (Apache-2.0).
- Joint Committee for Guides in Metrology. International Vocabulary of Metrology – Basic and General Concepts and Associated Terms (VIM). JCGM 200:2012, 3rd edition, 2012. URL https://www.bipm.org/documents/20126/2071204/JCGM_200_2012.pdf.
- Friedrich Leisch. Sweave: Dynamic Generation of Statistical Reports Using Literate Data Analysis. In *Compstat: Proceedings in Computational Statistics*, pages 575–580. 2002. doi: 10.1007/978-3-642-57489-4_89. URL https://doi.org/10.1007/978-3-642-57489-4_89.
- Donald MacKenzie. *Mechanizing Proof: Computing, Risk, and Trust*. The MIT Press, 2001. doi: 10.7551/mitpress/4529.001.0001. URL <https://doi.org/10.7551/mitpress/4529.001.0001>.
- João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pages 507–517, 2019. doi: 10.1109/MSR.2019.00077. URL <https://doi.org/10.1109/MSR.2019.00077>.

- Schema.org. ClaimReview and Claim, 2026. URL <https://schema.org/ClaimReview>. Vocabulary types ClaimReview and Claim, release 30.0 of 2026-03-19; definitions checked against the released JSON-LD artifact. Read 2026-08-27.
- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark, 2024. URL <https://arxiv.org/abs/2409.11363>.
- Eran Tal. Calibration: Modelling the measurement process. *Studies in History and Philosophy of Science Part A*, 65-66:33–45, 2017. doi: 10.1016/j.shpsa.2017.09.001. URL <https://doi.org/10.1016/j.shpsa.2017.09.001>.
- James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018. doi: 10.18653/v1/n18-1074. URL <https://doi.org/10.18653/v1/n18-1074>.
- Peter Todd. OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin. Blog post, September 2016. URL <https://petertodd.org/2016/opentimestamps-announcement>.
- David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or Fiction: Verifying Scientific Claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.609. URL <https://doi.org/10.18653/v1/2020.emnlp-main.609>.
- Boya Zhang, Alban Bornet, Anthony Yazdani, Philipp Khlebnikov, Marija Milutinovic, Hossein Rouhizadeh, Poorya Amini, and Douglas Teodoro. A dataset for evaluating clinical research claims in large language models. *Scientific Data*, 2025. doi: 10.1038/s41597-025-04417-x. URL <https://doi.org/10.1038/s41597-025-04417-x>.

A Conformance suite and fault injection

Conformance. The suite holds 25 fixtures: five addressing a table cell, four comparing a value, three each for prose anchors, two-sided correspondences and artifact availability, two each for a pointer and a passage, and one each for an unpinning artifact, a broken pin, and a claim offering no evidence. Each records its canonical expected output as an (outcome, reason, artifact validity) triple, so a case cannot pass by reaching the right verdict for the wrong cause. Assertions over them include that an absent pointer yields `extraction=absent` and never `comparison=mismatch`, and that an injected `TypeError` yields `error` and never `unchecked`.

Fault injection. Four faults are injected and the reported outcome recorded: a results file edited after drafting yields a broken pin with decisions against it marked non-authoritative; an absent extraction toolchain yields `unchecked` with reason `extractor_missing`; a registered plan edited after registration yields a broken pin on the plan artifact; a backend raising `TypeError` yields `error` with reason `backend_defect`. Each distinguishes a class the flattened view would otherwise merge.

The quote variant was run over an audit of sixteen published claims: 2,512 quotations against 125 content-addressed sources, of which the fifteen PDFs of the papers under audit carry 2,364. Every source carries a digest, so the document a failing quotation was checked against is byte-identical to the one the manuscript cited. Under poppler’s layout mode, 113 quotations did not resolve. One rater, who also wrote the quotations, attributed every one to extraction rather than to misquotation, each cause being recoverable from the recorded extractor output: 110 came from the mode’s preservation of page geometry, which severs every sentence crossing the gutter of a two-column paper, 2 from a text layer omitting a character the passage contains, and 1 from a hyphen at a line end that de-hyphenation cannot attribute. On this corpus a two-state checker therefore carries a false-accusation rate of 4.5%: it would have recorded all 113 as misquotations, each an accusation against the manuscript, and the corpus held no misquotation for it to find.

B Session status

The Claude Code mod reads the project's records at the end of each turn and draws them above the prompt, one row per command. `prereg` gives the plans frozen out of the plans present, `results` the runs, sealed inputs and bound claims the ledger holds, and `citations` the quotations pinned and whether the last full check resolved them. A second line appears under the prompt only when a record is out of step with what was registered.

```
knockout-epistasis-dynamics · prereg: 3/3 frozen          [-]
results: 13 runs, 8 sealed, 131 claims
citations: 18 pinned, not verified

----- knockout-epistasis -
> █

warning-demo · prereg: 1 changed                          [-]
results: 1 run, 0 sealed, 0 claims
citations: none pinned

----- warning demo -
> █

△ repro: 1 plan edited after freeze · 1 run, nothing sealed

proteome-mr-claim-audit · prereg: 1/1 frozen              [-]
results: 16 runs, 4 sealed, 0 claims
citations: 7,783 pinned, not verified

----- proteome-mr-survey -
> █
```

Figure 1: The state of a project's records as a session shows it. Top: the project audited in Section 5.2, at the commit audited, with every plan frozen and no warning. Middle: a project in which a frozen plan was edited and a run was recorded with no input sealed; the line under the prompt names both. Bottom: a literature survey in progress, with its plan frozen, no claim yet bound, and its pinned quotations not yet checked in this session. The label at the right of each prompt is the session's name.